

BUP WebSocket V2

Ziel

`bupws_v2` ist das neue WebSocket-Protokoll zwischen BTS und den BUP-Anzeigen/Tablets. Der wichtigste Unterschied zu V1 ist, dass der Server nicht mehr primär einen großen Turnier-/Event-Zustand verteilt, aus dem der Client seine Anzeige selbst rekonstruiert. Stattdessen sendet der Server gezielte Display-DTOs, die bereits auf den jeweiligen Client, dessen Feldzuordnung und dessen Anzeigestil zugeschnitten sind.

Das reduziert Netzwerkverkehr und verschiebt teure Zustandsberechnungen vom Anzeigegerät auf den Server. Das ist besonders relevant für schwache Display-Clients wie Raspberry Pi Zero 2.

Aktivierung auf Client-Seite:

```
https://<server>/bup/#btsh_e=<tournament>&display&dm_style=<style>&bupws_v2=1
```

Debug-Logging im Browser:

```
&bupws_v2_debug=1
```

Grundprinzip

V1:

- Client erhält einen großen Event-/Match-Zustand.
- Client sucht Court, Match, Teams, Scores und Settings selbst heraus.
- Client berechnet große Teile des Anzeigezustands lokal.
- Bei Änderungen wird häufig breiter neu gerendert.

V2:

- Server ermittelt pro Client den wirksamen Display-Zustand.
- Server sendet nur die für diese Anzeige relevanten Felder.
- Laufende Punktänderungen werden als kleines `display_score_update` gesendet.
- Der Client hält einen lokalen Cache und patcht nur betroffene DOM-Teile.
- Jede V2-Nachricht bekommt eine `message_id`; der Client bestätigt nach dem Rendern per ACK.

Payload-Typen

`display_state`

Vollzustand für eine Single-Court-Anzeige.

Wird gesendet bei:

- Erstverbindung
- Feldwechsel
- Wechsel des aktiven Matches
- Änderung von Anzeige-/Turnierdaten, die nicht durch ein Score-Update abgedeckt sind
- Fallback, wenn ein inkrementelles Update nicht sicher angewendet werden kann

Enthält unter anderem:

- `tournament` : Turnier-Key, Name, Logo-Version und optional Logo-Assets
- `display` : Client-ID, Hostname, Monitor-Label
- `court` : Feld-ID, Feldnummer, Label
- `match` : Match-ID, Status, Disziplin, Runde, Zählweise, Zeitdaten
- `teams` : Seiten, Teamnamen, Spieler, Gewinnerstatus
- `score` : aktuelle Punkte, gewonnene Sätze, fertige Sätze, Satzgewinner
- `service` : Aufschläger und Rückschläger inklusive Team-/Spielerindex
- `timers` : aktive Timerdaten
- `display_settings` : wirksame Anzeigeeinstellungen

`display_multi_state`

Vollzustand für Multi-Court-Anzeigen.

Wird verwendet für Anzeigen wie:

- `2court`
- `castall`
- `stream`
- `top+list`
- `teamscore`
- `tournament_overview`
- `tournament_overview_dm`
- `tournament_overview_dm_finals`

Der Payload enthält eine Liste `court_states`. Jedes Element ist im Kern ein `display_state` ohne erneut eingebettete Turnier-Assets. Dadurch werden Mehrfeldanzeigen initial vollständig versorgt, ohne Logo- und Turnierdaten pro Feld zu duplizieren.

Für Zweifeldanzeigen wählt der Server passend zur Feldzuordnung die relevanten benachbarten Felder aus, zum Beispiel 5 & 6.

`display_score_update`

Kleines inkrementelles Update für Punkt-, Aufschlag- und Timeränderungen.

Wird gesendet, wenn:

- das Match auf dem Client bereits bekannt ist
- sich kein struktureller Teil der Anzeige geändert hat
- keine neue Vollzustands-Nachricht nötig ist

Enthält gezielt:

- `court_id`
- `match_id`
- `status`
- `score`
- `service.server`
- `service.receiver`
- `timers.active_timer`
- optional `winner_side`
- optional `end_timestamp`

Der Client wendet dieses Update auf den zuletzt empfangenen `display_state` oder `display_multi_state` an. Danach ruft er den V2-Renderer im Patch-Modus auf.

`court_picker_state`

Zustand für nicht zugewiesene Tablets/Displays.

Wird verwendet, wenn ein Tablet oder Display keine Feldzuordnung hat. Der Client kann damit eine Feldauswahl anzeigen oder, bei Display-only-Ansichten, Hostname und Monitor-Label groß darstellen.

Netzwerk-Effizienz

Relevanzfilter pro Client

V2 sendet nicht mehr pauschal den gesamten Turnierzustand an jedes Display. Der Server bestimmt zuerst:

- welches Turnier aktiv ist
- welche Display-Einstellung gilt
- welches Feld oder welche Felder relevant sind
- ob der Client eine Single- oder Multi-Court-Anzeige ist
- ob nur ein Score-Update reicht oder ein Full-State nötig ist

Ein Display auf Feld 6 bekommt dadurch keine vollständigen Daten aller anderen Felder, außer der gewählte Anzeigestil braucht sie explizit.

Kleine Score-Updates statt Full-State

Bei normalen Punktänderungen reicht `display_score_update`. In beobachteten lokalen Logs lagen typische Größen ungefähr bei:

- `display_state`: ca. 1.8 KB
- `display_score_update`: ca. 0.45 KB

Das ist kein garantiertes Protokollmaß, weil Namen, Disziplinen und Settings variieren. Es zeigt aber die Größenordnung: reine Punktopdates sind typischerweise nur etwa ein Viertel eines Full-State-Payloads.

Noch wichtiger: V2 vermeidet bei Punktwechseln das erneute Senden von statischen Daten wie:

- Spielernamen
- Disziplin/Runde
- Feld-Metadaten
- Display-Settings
- Turnierlogo
- Logo-Farben

Payload-Deduplizierung

Der Server merkt sich pro WebSocket den zuletzt gesendeten Full-State und den zuletzt gesendeten Score-State.

Für `display_score_update` wird kein vollständiges `JSON.stringify` als Vergleichsschlüssel genutzt, sondern ein primitiver Cache-Key aus den tatsächlich relevanten Feldern:

- Punkte
- fertige Sätze
- Gewinnerseite
- Aufschlag/Rückschlag
- Timerdaten
- Match-/Court-ID
- Status

Wenn sich dieser Key nicht geändert hat, wird kein Update gesendet.

Turnier-Assets werden gecacht

Logo-Assets werden mit einer `logo_assets_version` versehen. Der Client hält Logo-URL und Farben im Speicher. Spätere Payloads können die Assets weglassen, solange sich die Version nicht ändert.

Das spart vor allem bei Multi-Court-Anzeigen unnötige Wiederholung von Turnier-/Logo-Metadaten.

Court-Batching auf Serverseite

Bei Score-Änderungen gruppiert der Server V2-Panels nach Court. Für mehrere Displays auf demselben Feld wird der relevante Kontext einmal berechnet und dann an die passenden Clients verteilt.

Multi-Court-Anzeigen werden getrennt behandelt. Wenn nur ein Feld betroffen ist und das Match bekannt bleibt, kann auch dort ein `display_score_update` für genau dieses Feld verschickt werden.

Modellvergleich Netzwerkverkehr

Die folgenden Werte sind Modellrechnungen. Sie beschreiben die Größenordnung und die Skalierung, ersetzen aber keine Messung in einer konkreten Halle.

Annahmen für das Modell:

- 8 aktive Felder
- 1 Feldanzeige pro Feld
- optional 1 zusätzliche Multi-Court-Anzeige
- pro Feld durchschnittlich 1 Punkt alle 10 Sekunden
- dadurch 0,8 Score-Ereignisse pro Sekunde in der Halle
- typischer V2-Full-State aus lokalen Logs: ca. 1,8 KB
- typisches V2-Score-Update aus lokalen Logs: ca. 0,45 KB
- typisches ACK zurück zum Server: grob 0,15 bis 0,25 KB

V2 bei einer Feldanzeige pro Feld:

- Downstream Score-Updates: $0,8 \text{ Ereignisse/s} * 0,45 \text{ KB} = \text{ca. } 0,36 \text{ KB/s}$
- Upstream ACKs: $0,8 \text{ Ereignisse/s} * \text{ca. } 0,2 \text{ KB} = \text{ca. } 0,16 \text{ KB/s}$
- Gesamt für reine Punktupdates: ca. 0,52 KB/s, also ca. 4,2 kbit/s

V2 mit zusätzlicher Multi-Court-Anzeige:

- Feldanzeigen: ca. 0,36 KB/s Downstream
- Multi-Court-Anzeige erhält ebenfalls relevante Score-Updates: zusätzlich ca. 0,36 KB/s Downstream
- ACKs für beide Anzeigegruppen: ca. 0,32 KB/s Upstream
- Gesamt für reine Punktupdates: ca. 1,04 KB/s, also ca. 8,3 kbit/s

Konservative V1-Untergrenze:

- Falls V1 bereits nur an das relevante Feld sendet und pro Punkt nur einen Full-State in der Größenordnung von 1,8 KB schickt:
- Downstream: $0,8 \text{ Ereignisse/s} * 1,8 \text{ KB} = \text{ca. } 1,44 \text{ KB/s}$
- V2 mit 0,36 KB/s Downstream spart in diesem Minimalmodell ca. 75 Prozent Downstream-Verkehr.

Typisch ungünstigere V1-Modelle:

- Wenn V1 bei jedem Punkt einen größeren Event-/Match-Zustand an alle 8 Feldanzeigen verteilt, skaliert der Traffic mit `Payloadgröße * Ereignisse/s * Anzeigen`.
- Bei 50 KB pro V1-Update: $50 \text{ KB} * 0,8 * 8 = \text{ca. } 320 \text{ KB/s}$, also ca. 2,6 Mbit/s
- Bei 100 KB pro V1-Update: $100 \text{ KB} * 0,8 * 8 = \text{ca. } 640 \text{ KB/s}$, also ca. 5,2 Mbit/s
- Bei 250 KB pro V1-Update: $250 \text{ KB} * 0,8 * 8 = \text{ca. } 1.600 \text{ KB/s}$, also ca. 13,1 Mbit/s

Vergleich zur V2-Modellrechnung:

- V2 mit nur Feldanzeigen: ca. 0,36 KB/s Downstream für Punktupdates
- V2 mit Feldanzeigen plus Multi-Court-Anzeige: ca. 0,72 KB/s Downstream für Punktupdates
- Gegenüber einem 100-KB-V1-Broadcast-Modell reduziert V2 den Downstream von ca. 640 KB/s auf ca. 0,36 bis 0,72 KB/s.
- Das entspricht grob einer Reduktion um 99,9 Prozent im Score-Update-Pfad.

Der wichtigste Punkt ist nicht nur die absolute Bandbreite, sondern die Skalierung: V1 kann mit der Anzahl der Displays und der Größe des Event-Zustands wachsen. V2 wächst bei Punktupdates primär mit der Anzahl der tatsächlich betroffenen Anzeigen und bleibt pro Update klein.

Weniger Arbeit auf den Anzeigegeräten

Server berechnet den Anzeigezustand

Der Server berechnet vorab:

- aktuelles Match pro Feld
- Score-Struktur für die Anzeige
- Aufschläger
- Rückschläger
- Satzgewinner
- Matchgewinner
- aktive Timer
- Matchdauer
- wirksame Display-Settings

Der Client muss diese Werte nicht mehr aus Presses und vollständigen Matchdaten rekonstruieren. Er rendert weitgehend DTO-Felder.

Native V2-Renderer statt Legacy-Event-Aufbau

Für migrierte Styles ruft der Client direkt native V2-Renderer auf, zum Beispiel:

- `render_v2_tournamentcourt_display_state`
- `render_v2_2court_display_state`
- `render_v2_onscourt_display_state`
- `render_v2_teamcourt_display_state`
- `render_v2_tournament_overview_dm_display_state`

Nur noch nicht-native Fallback-Pfade bauen aus V2 einen Legacy-Event-Zustand. Ziel ist, diese Fallbacks pro Style zu entfernen.

DOM-Patching statt Vollrender

Viele V2-Renderer halten interne Caches auf DOM-Elemente:

- Score-Elemente
- Spielernamen
- Meta-Zeilen
- Timerfelder
- Service-Markierungen
- Court-Zeilen bei Multi-Court-Views

Bei `display_score_update` werden nur die betroffenen Felder geändert. Beispiele:

- `tournamentcourt` : Punkte, Satzstand, Aufschlag, Timer
- `2court` : nur betroffene Hälfte oder betroffene Score-/Timerbereiche
- `tournament_overview_dm` : nur die betroffene Zeile oder Score-Zellen

Dadurch muss der Browser weniger Layout, weniger Paint und weniger DOM-Erzeugung leisten.

Weniger lokale Suche

In V1 musste der Client oft:

- das richtige Court-Objekt suchen
- das passende Match suchen
- Teams aus Setup/Match extrahieren
- Score aus `network_score` oder Presses ableiten
- Aufschlag/Rückschlag bestimmen

In V2 ist diese Suche bereits serverseitig erledigt. Der Client arbeitet mit direkten DTO-Feldern wie `court`, `match`, `teams`, `score` und `service`.

Weniger JSON-Arbeit im heißen Pfad

V2 reduziert wiederholte große `JSON.stringify`-Vergleiche und tiefe Kopien im Client. Bei schnellen Punktfolgen wird nur ein kleines Update in den bestehenden Cache gemerged.

Das ist auf Desktop-Systemen kaum spürbar, auf Pi Zero 2 aber relevant, weil dort JSON-Parsing, DOM-Neubau und Layout-Reflow schnell sichtbar werden können.

Modellvergleich Gerätearbeit

Auch die Entlastung der Anzeigegeräte lässt sich nur modellhaft angeben, weil sie von Browser, Displaystil, Namenlängen, Timeranzeige und Hardware abhängt. Die Richtung ist aber eindeutig.

Bei einem V1-artigen Update muss der Client typischerweise:

- einen größeren JSON-Payload parsen
- aus dem Event-Zustand das eigene Feld suchen
- das laufende Match suchen
- Score und Sätze aus Match-/Pressdaten ableiten
- Aufschläger und Rückschläger bestimmen
- Timerdaten rekonstruieren
- den Anzeigezustand neu zusammenbauen
- große Teile des DOM neu rendern oder neu layouten

Bei einem V2-Score-Update muss der Client typischerweise:

- einen kleinen JSON-Payload parsen
- `match_id` und `court_id` gegen den lokalen Cache prüfen
- Score, Service und Timer in den lokalen DTO-Cache übernehmen
- gezielt die vorhandenen DOM-Knoten patchen

Modellhafte Einsparung beim JSON-Parsing:

- V2-Score-Update gegenüber V2-Full-State: ca. 0,45 KB statt 1,8 KB, also ca. 75 Prozent weniger JSON-Daten
- V2-Score-Update gegenüber einem 50-KB-V1-Event: ca. 99,1 Prozent weniger JSON-Daten
- V2-Score-Update gegenüber einem 100-KB-V1-Event: ca. 99,55 Prozent weniger JSON-Daten
- V2-Score-Update gegenüber einem 250-KB-V1-Event: ca. 99,82 Prozent weniger JSON-Daten

Modellhafte Einsparung bei Berechnungen:

- Court-/Match-Auswahl: entfällt weitgehend auf dem Client
- Aufschlag-/Rückschlagberechnung: wird serverseitig geliefert
- Satzgewinner/Matchgewinner: wird serverseitig vorbereitet
- Timerzustand: wird serverseitig als DTO geliefert
- Anzeige-Settings: werden als wirksame Settings geliefert

Modellhafte Einsparung beim Rendering:

- V1/Vollrender: häufig kompletter Anzeigebaum oder große Teilbäume
- V2/Patch: häufig nur Score-Texte, Aufschlagmarkierung, Timer und wenige Statusklassen
- Bei Multi-Court-Views kann V2 statt der ganzen Anzeige nur die betroffene Zeile oder Court-Hälfte aktualisieren.

Für ein Pi-Zero-2-Display bedeutet das praktisch:

- weniger CPU-Spitzen bei schnellen Punktfolgen
- weniger Garbage Collection durch weniger kurzlebige JS-Objekte
- weniger Layout- und Paint-Arbeit im Browser
- weniger Flackern, weil vorhandene DOM-Elemente stehen bleiben
- stabilere Framerate bei animierten oder großen Anzeigen

Eine vorsichtige technische Aussage ist: Im heißen Score-Update-Pfad sinkt die clientseitige Datenmenge typischerweise um 75 Prozent gegenüber einem kleinen Full-State und um deutlich über 99 Prozent gegenüber einem großen V1-Event-Payload. Die lokale Anzeige-Arbeit sinkt von "Zustand rekonstruieren und Anzeige neu aufbauen" auf "kleines DTO anwenden und wenige DOM-Knoten patchen".

ACKs und Statusüberwachung

Jede gesendete V2-Nachricht erhält:

- `message_id`
- `sent_ts`

Nach dem Rendern antwortet der Client:

```
{
  "type": "display_rendered",
  "message_id": "...",
  "payload_type": "display_score_update",
  "ok": true,
  "render_ms": 12
}
```

Der Server nutzt ACKs für:

- Online-/Offline-Erkennung
- Timeout-Erkennung
- durchschnittliche Roundtrip-Zeit der letzten 5 Minuten
- letzte ACK-Zeit
- optionales Fluten oder Nicht-Fluten der Admin-Warteanzeige bei schnellen Score-Updates

Wichtig: Die relevante Netzwerkkenzahl ist `roundtrip_ms`, also Zeit von Server-Senden bis Server-ACK-Empfang. `render_ms` ist nur die vom Client gemeldete lokale Renderdauer.

Full-State-Fallbacks

Ein `display_score_update` wird nur verwendet, wenn der Client denselben Match-Kontext kennt. Wenn sich das Match geändert hat oder der Server nicht sicher ist, wird stattdessen ein neuer Full-State gesendet.

Typische Fallback-Gründe:

- neues Match auf dem Feld
- Feld ist leer geworden

- Display-Einstellung wurde geändert
- Multi-Court-Liste hat sich strukturell geändert
- Server erkennt abweichende `match_id`
- Timer-/Statuswechsel benötigt neue Anzeigegrundlage

Das hält die inkrementellen Updates sicher. Lieber einmal mehr Full-State als ein Score-Update auf eine falsche lokale Anzeige anwenden.

Leeres Feld und nicht zugewiesene Displays

Wenn ein Display keinem Feld zugeordnet ist, zeigt V2 standardisiert eine Identifikationsseite:

- Hostname
- Monitor-Label

Wenn ein Feld zugeordnet ist, aber kein Match läuft, können normale Display-Styles eine leere Feld-/Logo-Ansicht rendern. Diese Ansicht kommt ebenfalls aus V2-Daten und muss nicht aus einem vollständigen Event-Zustand abgeleitet werden.

Erwartete Wirkung in einer Halle

Bei einer Halle mit mehreren Feldern entstehen die größten Gewinne an drei Stellen:

1. Punktupdates werden klein. 2. Displays bekommen nur relevante Court-/Matchdaten. 3. Schwache Clients patchen vorhandene DOM-Knoten statt die Anzeige komplett neu aufzubauen.

Bei 8 Feldern und mehreren Anzeigen sinkt dadurch nicht nur die Bandbreite, sondern auch die CPU-Last auf den Displays. Besonders Pi Zero 2 profitiert, weil die Arbeit von "Event interpretieren und Anzeige neu berechnen" zu "kleines DTO anwenden und wenige DOM-Knoten ändern" wird.

Grenzen und offene Optimierungsmöglichkeiten

V2 ist bereits deutlich effizienter als V1, aber weitere Optimierungen sind möglich:

- `display_score_update` könnte optional noch weiter in reine Punkte-Updates und Timer-Updates getrennt werden.
- Timerdaten könnten nur gesendet werden, wenn sich timerrelevante Informationen wirklich ändern.
- Für alle Styles sollte der Legacy-Fallback vollständig entfernt werden.
- Render-Caches können pro Style noch feiner werden, etwa halbe Anzeige bei `2court` oder einzelne Zeile bei Multi-Court-Views.
- Debug-Logs sollten im Produktivbetrieb aus bleiben.
- Ein optionaler Pi-Agent könnte später WLAN-Qualität, Signalstärke und Systemlast direkt melden. Per HTTPS/WebSocket alleine sind diese Informationen im Browser nicht zuverlässig verfügbar.

Zusammenfassung

V2 macht aus der bisherigen breiten Event-Verteilung ein displayorientiertes DTO-Protokoll. Der Server sendet weniger, gezielter und bereits vorverarbeitet. Die Clients müssen weniger rechnen, weniger suchen und weniger neu rendern.

Die Effizienzgewinne entstehen nicht durch eine einzelne Maßnahme, sondern durch die Kombination aus:

- serverseitiger Vorberechnung
- kleinen inkrementellen Score-Updates
- Payload-Deduplizierung
- Turnier-Asset-Caching
- nativen V2-Renderern
- DOM-Patching
- Render-ACKs für Überwachung und Diagnose